

Software Engineering Final Exam Preparation

Topics : Opensource Software License– Software Cost Estimation
(9 Questions)

1. Opensource Software License (2 short answers)
2. Software Testing (4 main questions, 1 short answer)
3. Software Integration (1 short answer)
4. Project Management (1 main question, 1 short answer)
5. Software Cost Estimation (2 main questions, 1 short answer)

Opensource Software License (2 short answers) :

- If the GNU open source software is integrated with the software that currently the team is developing. When this software is done, can the team use Apache license for their own software?

It depends on the version of GNU General Public License(GPL) and the Apache License.

1. Apache License 2.0 is compatible with GNU GPL version 3

This means the code can be combined under GPLv3 and Apache License 2.0, but the resulting combined software must be licensed under GPLv3.

Therefore, the team cannot solely use the Apache license for this software.

2. GPLv1 and GPLv2 are not compatible with Apache License 2.0

This will cause licensing conflict.

3. The Apache Software Foundation(ASF) itself generally requires projects under its umbrella to be distributed under the Apache License 2.0. They often avoid GPLv3 because GPLv3 also requires the project to also be under GPLv3. These 2 requirements conflict each other in terms of licensing policy.

In conclusion, mostly the software from GNU GPL are not used cooperatively with Apache licensing because of their policy conflicts, however, the only case that these 2 licenses can be working together is that the software integrators use Apache License 2.0 and GPLv3 for their own software.

Cheat sheet?

table with every software licenses(rows) with the headers(columns) : Compatible, Not Compatible, Compatible in some cases, version that they are compatible

- What is the meaning of the figure below?



CC - Creative Commons License

BY - Attribution (require anyone who uses, distributes, or builds upon the work to give appropriate credit to the original creator)

NC - Non Commercial

Therefore, the meaning of the figure is Creative Commons Attribution-NonCommercial

- Do the source code's owners of open source software license take responsibility on their code if another programmer integrates these code to a new project?

No, most open-source software licenses contain explicit disclaimers of warranty and liability.

Software Testing (4 main questions, 1 short answer)

Check lotto no. application

Design the test case including the testing input for a **black-box** and **white-box testing**. In this application, users can input only two-digit no. from 00 to 99 into the application to check whether the input no. matches with one of the winning numbers. For example, if the winning no. are 03, 18, 46, 69, 84, then the input no. must be matched to one of these winning no. to win the lotto.

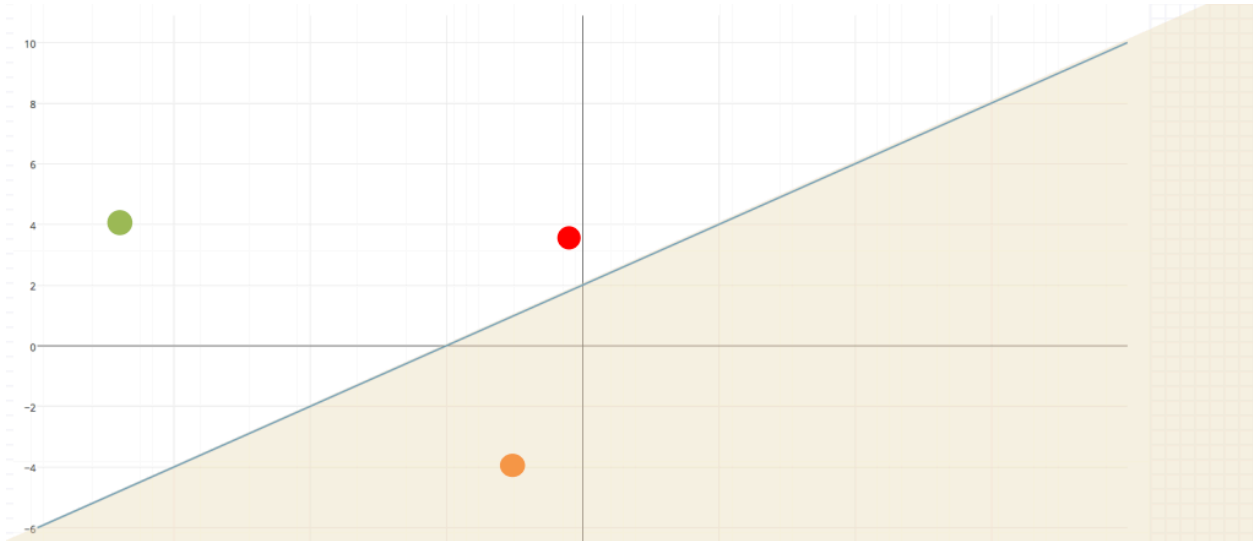
Black Box Testing (Functional Testing)

	Test Cases	Test data	Expected Result	Program Result	Note
Valid (Winning)	2 digit numbers from 0-99 (Matched)	03, 18, 46, 69, and 84	T	T	Pass
Valid (Losing)	2 digit numbers from 0-99 (Unmatched)	00, 01, 99, 15, 7, etc.	F	F	Pass
Invalid	Characters and special characters	“A”, “B”, “*”, “\$”, etc.	Reject	Reject	Not Pass
Invalid	1 or 3+ Digit Numbers	1, 5, 100, 2000, 9999, etc.	Reject	F	Pass
Invalid	Floating Numbers	11.1, 23.5, 15.01, etc.	Reject	Reject	Not Pass
Invalid	null	null	Reject	Reject	Not Pass

White Box Testing (Structural Testing)

Test Case	Test Data	Expected Result	Program Result	Note
When i == winning number	03	returns “Win!”	returns “Win!”	Pass
When i == losing number	00	return “Lose”	return “Lose”	Pass
When i > 99	123	Failed	Failed	Not Pass
When i < 00	-1	Failed	Failed	Not Pass
When i == char or string	“XX”, “A”	Failed	Failed	Not Pass

Design the **Black-box** and **White-box** Testing for the Graph Below



The application can classify whether the point (x,y) is inside the yellow area. If $y < x+2$, the point (x,y) is inside the yellow area. The input of this application is the point or coordinate (x,y) . x and y are integers. The output of this application is 'true' when the point (x,y) is inside the yellow area and 'false' when the point is outside the yellow area.

Black Box Testing

	Test Case	Test Data	Expected Result	Program Result	Note
Valid (Inside the area)	(x, y) where $y < x + 2$	(1, 1)	T	T	Pass
Valid (Outside the area)	(x, y) where $y \geq x + 2$	(1, 3)	F	F	Pass
Invalid	Characters and special characters	"A", "\$"	Reject	Reject	Not Pass
Invalid	Miss 1 element in a tuple	(, 2)	Reject	Reject	Not Pass
Invalid	Null	-	Reject	Reject	Not Pass

White Box Testing

Test Case	Test Data	Expected Result	Program Result	Note
(x, y) where $y < x + 2$	(1,1)	T	T	Pass
(x, y) where $y \geq x + 2$	(1,4)	F	F	Pass
Not in (x,y) form: characters, special characters, integers, floating numbers	“A”, “&”, 1, 1.1	Reject	Reject	Not Pass

Software Integration (1 short answer)

▪ Problem before DevOps

1. Wall of Conflict (between Dev and Ops)
2. Slow Deployment Frequency and Long Lead Times
3. Low Reliability and Customer Responsiveness
4. Sequential and Disconnected Processes
5. Pain Points Across Roles (Dev, Ops, Test Engineers, Product Managers, System Administrators, Customer Service, and End Users)

▪ DevOps Process & Tools

DevOps Process:

- Plan → Code → Integrate → Test → Release → Deploy → Operate → Monitor

Key Concepts and Practices

- 5 Cs of DevOps:
 - Continuous Integration (CI)
 - Continuous Testing
 - Continuous Delivery (CD)
 - Continuous Monitoring
 - Continuous Deployment
 - Continuous Monitoring
- Automation (Maven → JUnit → SonarQube → Jenkins → Jmeter)
- Infrastructure as Code (IaC) (Ansible, Terraform, Chef, Puppet)
- Containerization (Docker)
- Software Configuration Management (SCM)
- KPIs (Key Performance Indicators) - Metrics to measure the efficiency of DevOps
 - Time to Production, Deployment Frequency, Average Lead Time, Production Failure Rate, and Mean Time To Recover (MTTR)

Tools:

- Collaborate : Jira, Trello, Git, Github
- Build : Maven, Gradle, Docker
- Test : Selenium, JUnit, JMeter
- Deploy : Jenkins, Ansible, Kubernetes, Docker
- Run/Monitor : Prometheus, Grafana, Elastic Stack

▪ DevOps Solution for one scenario

Scenario:

Bridging the Gap between Development and Operations for Faster, More Reliable Releases

Problem Before DevOps:

Slow, manual deployments causing "it works on my machine" issues, blame games between Dev and Ops, and delayed features for customers.

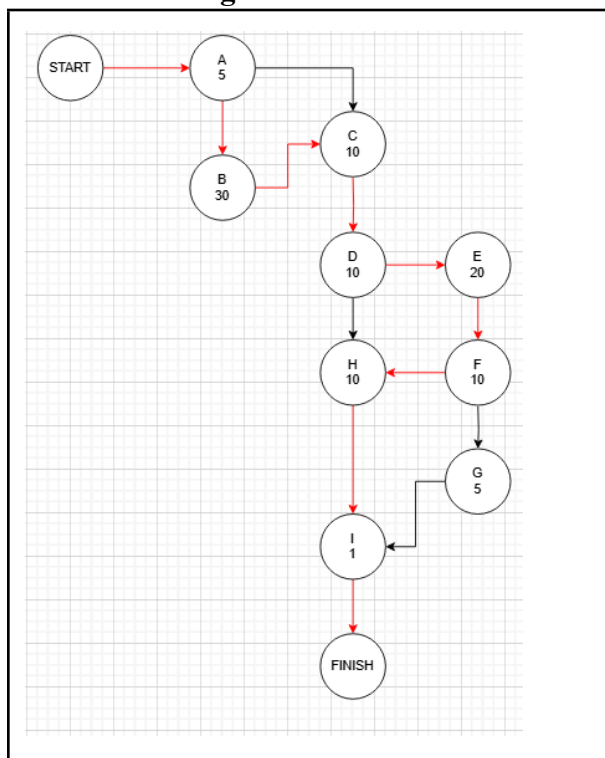
DevOps Solution:

- **Unified Goal & Collaboration:** Dev and Ops teams work together with shared goals, using tools like Jira for task tracking
- **Continuous Integration (CI):** Developers frequently commit code, triggering automated builds and tests to catch issues early
- **Infrastructure as Code (IaC) & Containerization:** Infrastructure is defined as code (e.g., Terraform), and apps are packaged in containers (e.g., Docker), ensuring consistent environments and eliminating configuration drift
- **Continuous Delivery/Deployment (CD):** Validated applications are automatically deployed to various environments, enabling daily or multiple daily releases
- **Continuous Monitoring:** Real-time tools (e.g., Prometheus) track application performance, allowing quick issue identification and resolution

Project Management (1 main question, 1 short answer)

	Task	Duration	Start	End	Pred
A	Identify Interviewee	5	3-Sep-2025	7-Sep-2025	
B	Conduct Interviews	30	10-Sep-2025	19-Oct-2025	A
C	Review Documentation	10	22-Oct-2025	02-Nov-2025	A,B
D	Analyze Data	10	5-Nov-2025	16-Nov-2025	C
E	Draft Preliminary Report	20	19-Nov-2025	14-Dec-2025	D
F	Obtain Feedback on Draft Report	10	17-Dec-2025	28-Dec-2025	E
G	Write Final Report	5	31-Dec-2025	4-Jan-2025	F
H	Prepare Presentation	10	31-Dec-2025	11-Jan-2025	D, F
I	Make Final Presentation	1	14-Jan-2025	14-Jan-2025	H, G

• Draw Pert diagram



• Find critical path and critical duration

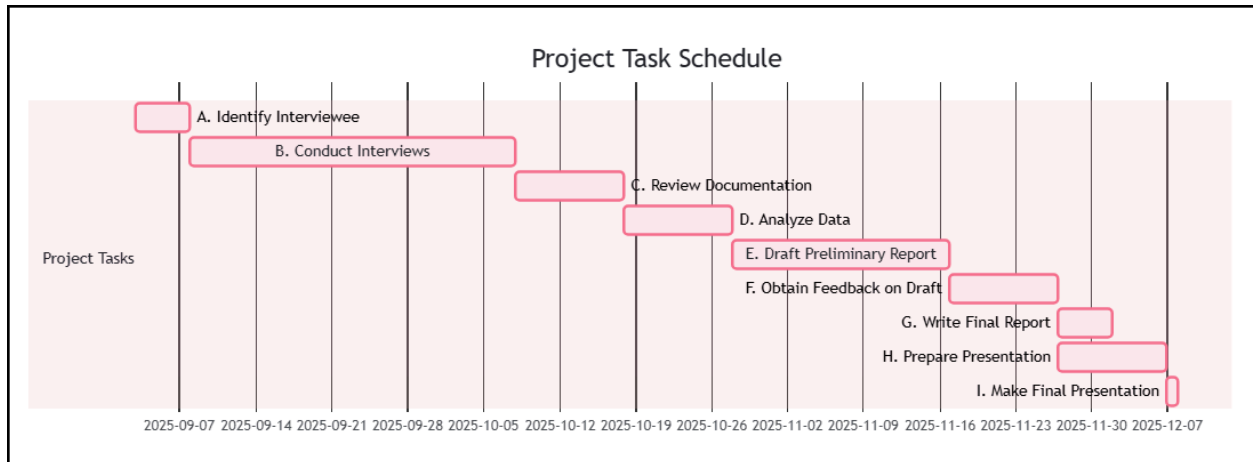
Critical Path = A B C D E F H I = $5+30+10+10+20+10+10+1 = 96$

- Find slack time of each path

Slack Time for each path

1. $A C D H I = 5+10+10+10+1 = 36$
2. $A C D E F G I = 5+10+10+20+10+5+1 = 61$
3. $A C D E F H I = 5+10+10+20+10+10+1 = 66$
4. $A B C D H I = 36+30 = 66$
5. $A B C D E F G I = 61+30 = 91$
6. $A B C D E F H I = 66+30 = 96$ (Critical Path)

- Draw Gantt chart



Software Cost Estimation (2 main questions, 1 short answer)

1. COCOMO II Effort Equation

As an example, a project with all Nominal Cost Drivers and Scale Drivers would have an M of 1.00 and exponent is equal 1.0997. Assuming that the project is projected to consist of 8,000 source lines of code, **Find COCOMO II estimation?**

The effort is estimated using the formula:

$$PM = A \times SIZE^B \times M$$

where :

- PM = Effort in Person-Month
- A = Constant (typically 2.94)
- SIZE = Size of the software in Kilo Lines of Code (KLOC)
- B = Exponent determined by Scale Factors
- M = Effort Adjustment Factor (Multiplier), the product of all Cost Drivers

The given information are as followed:

- M = 1.00
- B = 1.0997
- Project Size (SLOC) = 8,000
- A (Constant) = 2.94 (Based on lecture 15)

First, we need to convert from Project Size (SLOC) into Size of the software in Kilo Lines of Code (KLOC):

$$SIZE (KLOC) = \frac{8,000 SLOC}{1000} = 8 KLOC$$

Now, we know every necessary element in the effort estimation formula:

$$\begin{aligned} PM &= A \times SIZE^B \times M \\ PM &= 2.94 \times 8^{1.0997} \times 1 \\ PM &\approx 28.94 \end{aligned}$$

Therefore, the COCOMO II Estimation is approximately 28.94 Person-Months

2. Effort Adjustment Factor (referring from 1)

The Effort Adjustment Factor in the effort equation is simply the product of the effort multipliers corresponding to each of the cost drivers for your project.

For example, if your project is rated Very High for Complexity, and Low for Language Experience, and all of the other cost drivers are rated to be Nominal

Find the multiplier (M) and find COCOMO II estimation?

Cost Drivers	Ratings					
	Very Low	Low	Nominal	High	Very High	Extra High
Product attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
Personnel attributes						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project attributes						
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

Figure COCOMO II Cost Driver Multiplier Table

First, we need to find the Multiplier (M) which is the product of all Effort Multipliers. Since all others are Nominal, which are set to 1.00, we only need to multiply only the specific cost drivers provided

We know that the project is rated Very High for Complexity, Low for Language Experience. Therefore:

$$\begin{aligned}
 M &= CPLX \times LTEX \times 1(\text{Other Nominal Cost Drivers}) \\
 M &= 1.30 \times 1.07 \times 1 \\
 M &= 1.391
 \end{aligned}$$

Next, we further clarify COCOMO II Estimation, and again, the estimation formula comes:

$$PM = A \times SIZE^B \times M$$

But based on what we know right now, there are only Multiplier (M) and the constant A. The other 2 unknowns are SIZE and Exponent (B), so we are referring to them from (1). The the information to solve the COCOMO II estimation are:

- $A = 2.94$ (Typical as lecture 15)
- $SIZE = 8$ KLOC (From 1)
- $B = 1.0997$ (From 1)
- $M = 1.391$

Now, we continue doing the estimation:

$$\begin{aligned} PM &= A \times SIZE^B \times M \\ PM &= 2.94 \times 8^{1.0997} \times 1.391 \\ PM &\approx 40.25 \end{aligned}$$

Therefore, COCOMO II estimation is approximately 40.25 Person-Months.

In conclusion, $M = 1.391$ and $PM = 40.25$

CHEAT SHEET must HAVE:

1. Opensource Software License

License (Row)	Compatible With (Examples)	Not Compatible With (Examples)	Compatible In Some Cases (Conditions)	Version for Compatibility
Apache License 2.0	- MIT License - BSD License - Other Permissive licenses	GNU GPLv2 (and earlier)	GNU GPLv3: When you combine Apache 2.0 code into a GPLv3 project, a portion of the code still retains its Apache License. However, the combined work must then be distributed under the terms of the GPLv3.	Apache 2.0 with GPLv3
GNU GPLv3	- Apache License 2.0 - GNU AGPLv3 (explicitly compatible) - GNU LGPLv3 - Other GPLv3-compatible licenses	GNU GPLv2 (and earlier) - Apache License 2.0 (if the combined work must remain Apache 2.0)	Other Copyleft Licenses: Generally incompatible unless the other license has explicit relicensing clauses or the FSF declares specific compatibility.	GPLv3 with Apache 2.0, AGPLv3, LGPLv3
Creative Commons Licenses	- Other CC licenses (depending on specific clauses like ShareAlike, NoDerivatives)	Software source code licenses (e.g., GPL, MIT) for direct code integration.	Content Bundled with Software: If content (e.g., images, text) is licensed under CC and bundled with software, the CC license applies to the content, and the software's license applies to the code.	Varies by specific CC license (e.g., CC BY, CC BY-NC)
MIT License	- Apache License 2.0 - BSD License - GNU GPLv2, GPLv3, LGPL, AGPL (can be included in these projects, but the combined work takes on the copyleft license) - Almost all other licenses	None (Highly Permissive)	Every Case : Highly Permissive, as it imposes minimal restrictions)	All versions generally compatible
Mozilla Public License (MPL)	- Apache License 2.0 - MIT License - BSD License	GNU GPL (v2, v3) for older MPL versions (e.g., MPL 1.1)	GNU GPLv2 and GPLv3 (for MPL 2.0): MPL 2.0 is designed to be compatible with GPLv2 and GPLv3, allowing code under MPL 2.0 to be combined with GPL-licensed code. However, the MPL-licensed code remains under MPL, and modifications to MPL files must remain MPL.	MPL 2.0 with GPLv2, GPLv3